Reproducible Performance Measurement and Analysis for High-Performance Computing Applications

HPDC Tutorial

July 20, 2025

Stephanie Brink, David Boehme, Ian Lumsden,
Dewi Yokelson, Michela Taufer, and Olga Pearce

LLNL-CFPRES-2202755
This work was performed under the auspices of the U.S. Department of Energy by Lawrence Livermore National Laboratory under contract DE-AC52-07NA27344. Lawrence Livermore National Security, LLC

THE UNIVERSITY OF TENNESSEE KNOXVILLE

Lawrence Livermore National Laboratory

Tutorial Presenters

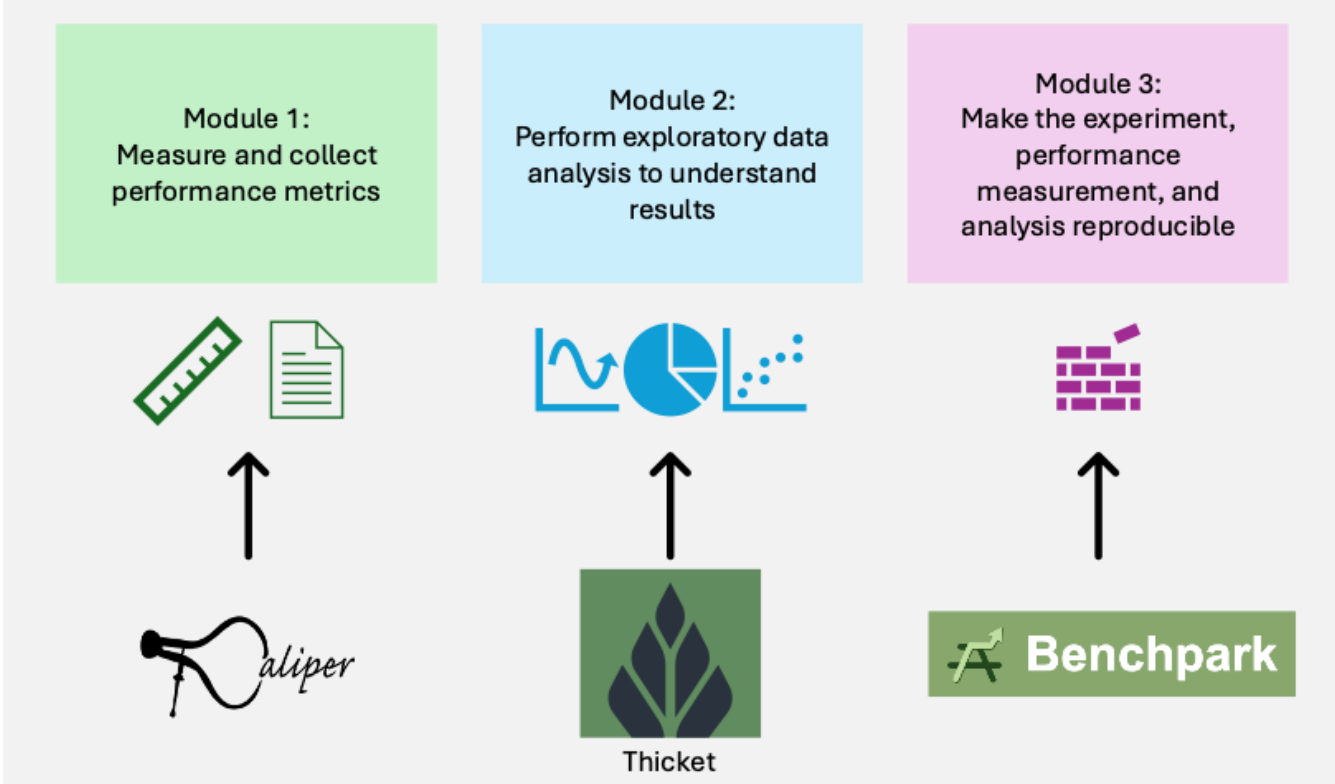
Stephanie Brink
LLNL

David Boehme
LLNL

Ian Lumsden
UTK

Overall Tutorial Agenda (approximate)

Welcome and Introduction	15 min
Module 1: Performance measurement with Caliper	45 min
Module 2: Performance analysis with Thicket	30 min
Break	30 min
Module 2 (cont'd): Performance analysis with Thicket	20 min
Module 3: Reproducible experiments with Benchpark	60 min
Q&A and Wrapup	10 min



*Hands on component to accompany each module

Overall Tutorial Materials

Find these slides and associated scripts here:

- Caliper: https://software.llnl.gov/Caliper/ under “Materials”
- Thicket: https://thicket.readthedocs.io/en/latest/tutorial_materials.html
- Benchpark: https://software.llnl.gov/benchpark/example-workflow.html

We also have a channel on Spack slack.

You can join here:

slack.spack.io

Join the **#benchpark-support** channel!

You can continue to ask questions here after the tutorial has ended.

Tutorial Instances: http://bit.ly/4kGQDlc

- We have an AWS instance for the hands-on component of this tutorial
- The instance provides:
  - Pre-installed Thicket, Caliper, and Benchpark and required dependencies
  - Caliper source code demos
  - Thicket Jupyter notebooks and datasets for performance analysis

When logging in to the instance:

- Please use a unique username to avoid resource allocation conflicts
  - First initial followed by last name (e.g., jdoe)
- PW: hpctutorial25

Performance Measurement with Caliper

David Boehme

Performance Analysis with Thicket I

Stephanie Brink

30 Min Break

Performance Analysis with Thicket II

Stephanie Brink

Reproducible Experiments with Benchpark

Stephanie Brink

We benchmark HPC systems for a variety of reasons

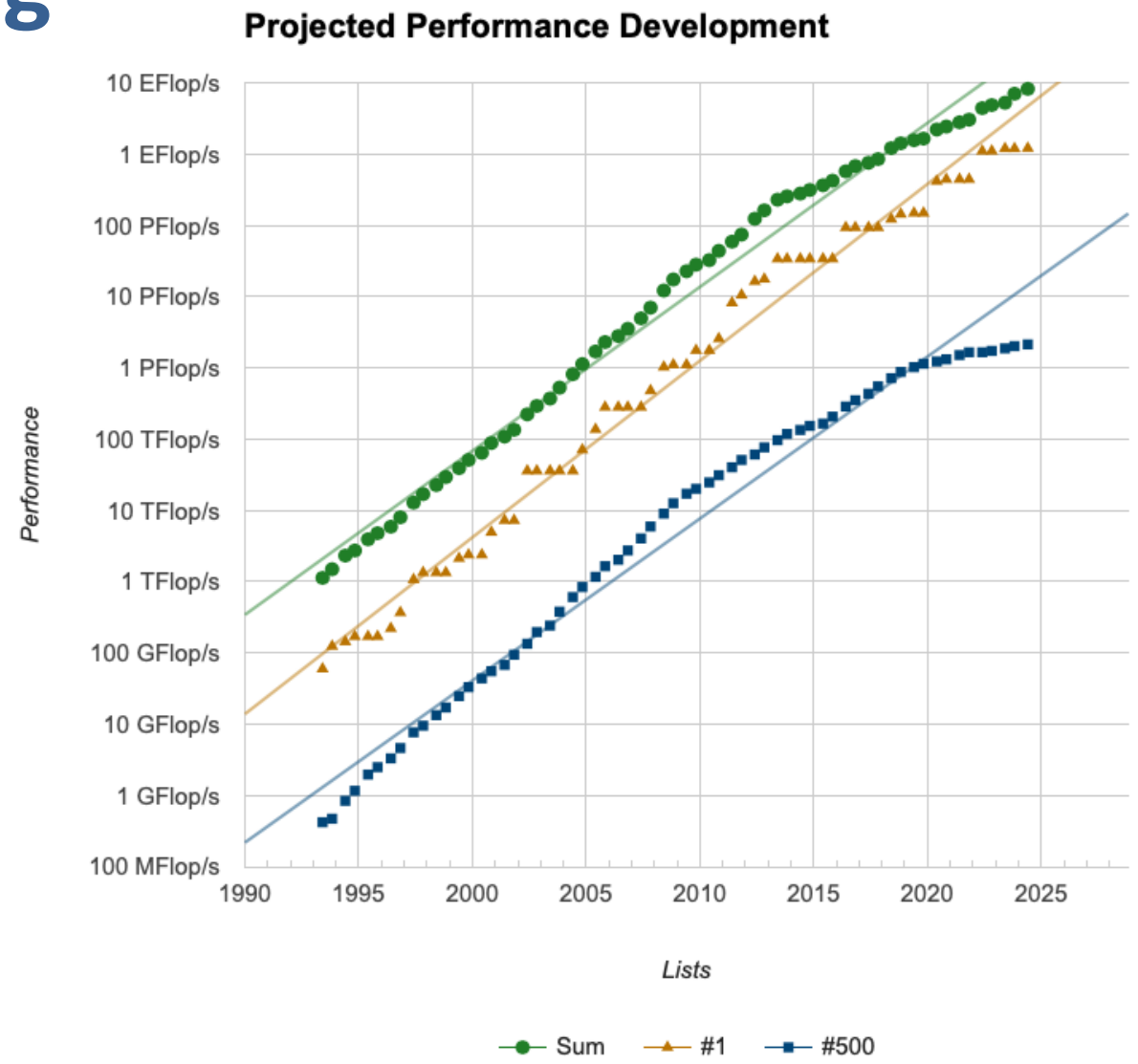
- Procurements
  - *Communicate* datacenter workload to vendors
  - Co-design systems, monitor progress
  - System acceptance (contractual specification)
- Validation of software stack, tools
  - Compilers
  - Debuggers
  - Correctness tools
  - Performance tools
- Research
  - Programming models
  - Computational methods
  - Performance across architectures

Many stakeholders, communication is key for collaboration

Source: https://www.top500.org/statistics/perfdevel/

Benchmarking is challenging

- What are we trying to characterize?
- Are we capturing the best the system can do?
- Is something else impacting performance?
- Did we build and run the code in the *optimal* and *reproducible* way?



HPC benchmarks run on diverse HPC systems

Lawrence Berkeley Nat'l Lab
AMD Zen + **NVIDIA**

F u g a k u
RIKEN **Fujitsu** ARM a64fx

Lawrence Livermore Nat'l Lab
IBM Power9 + **NVIDIA**

Lawrence Livermore Nat'l Lab
AMD Zen + **Radeon**

Oak Ridge National Lab
AMD Zen + **Radeon**

Argonne National Lab
Intel Xeon + Xe

- Benchmark source code
  - Abstraction (OpenMP, RAJA, Kokkos)
  - Hardware-specific (CUDA, ROCm)
- Optimized code for the CPU and GPU
  - Must make effective use of the hardware
  - Can make 10-100x performance difference
- Rely heavily on system packages
  - Need to use optimized communication and MPI libraries that come with machines

Writing benchmark source code is only the beginning

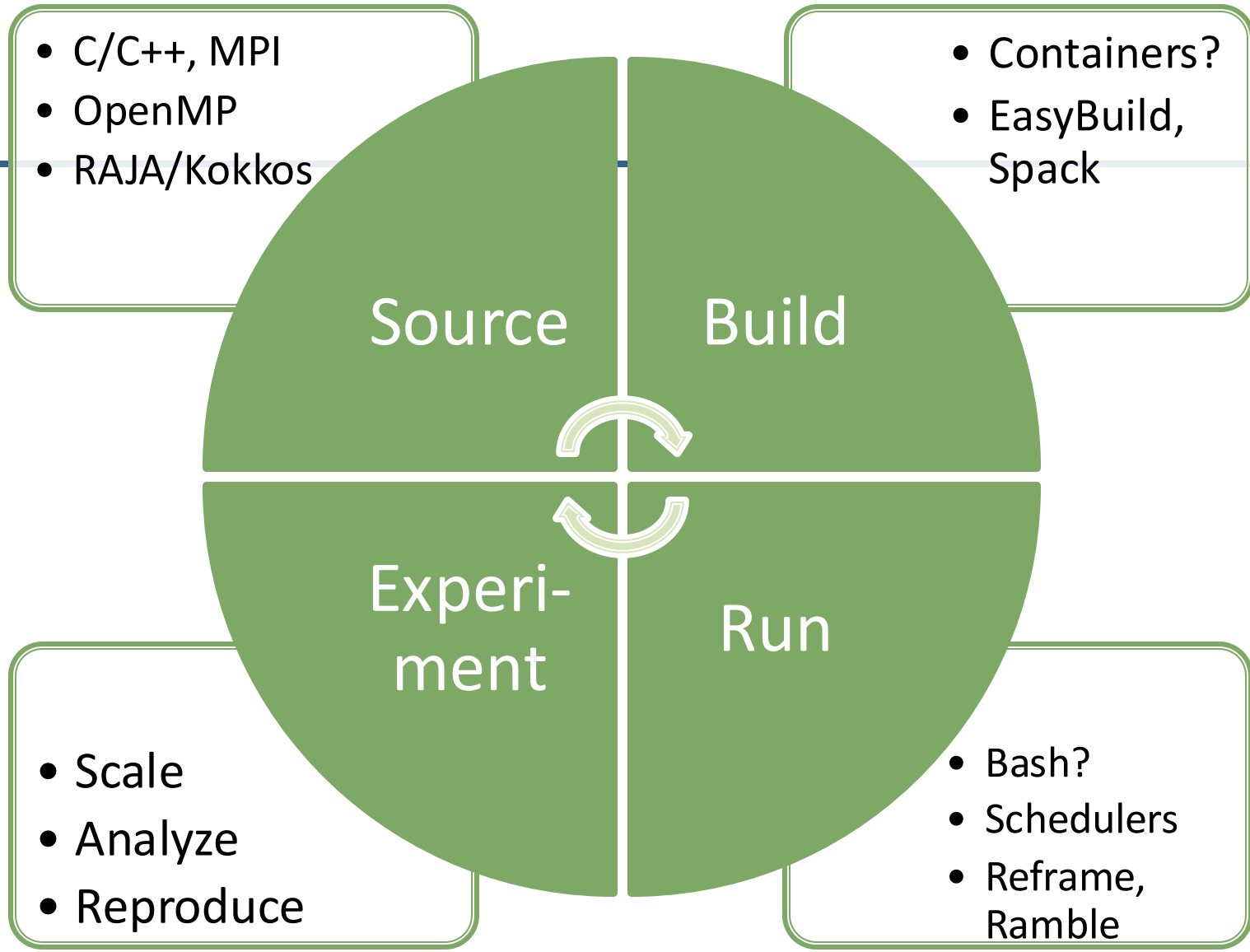
State of the practice:
HPC system benchmarking is manual!

- **Building** on each system is different, porting the builds to new systems is manual
- **Running** on each system is different, porting run scripts to new systems is manual
- Systems keep **changing**, requiring updates to how we build and run benchmarks
- **Triggerinåçg** builds and runs is manual: benchmark results don't stay up to date
- **Performance analysis** of results is manual

Communicate technical specification via code, documentation, white papers

HPC Benchmarks are HPC Software

- Portability
- Maintenance
- Testing/CI
- Verification
- Reproducibility



All components must work *for your system*, focus on explainable *performance*

Benchpark enables complete specification of HPC benchmarks

Infrastructure-as-code benchmark specification codifies:

- Benchmark build and run instructions
- **HPC Systems**
- **HPC Experiments**

github.com/llnl/benchpark

Leverage advances in HPC automation

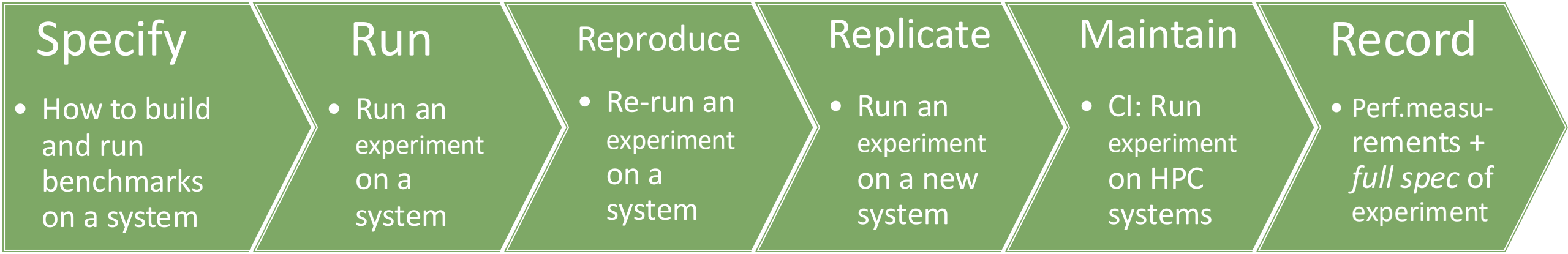
- Source code
- Build specification
- Run specification
- CI

Ramble

Spack

Every part of the specification codified: use to communicate, automate

Benchpark enables *reproducible* specifications of benchmarks



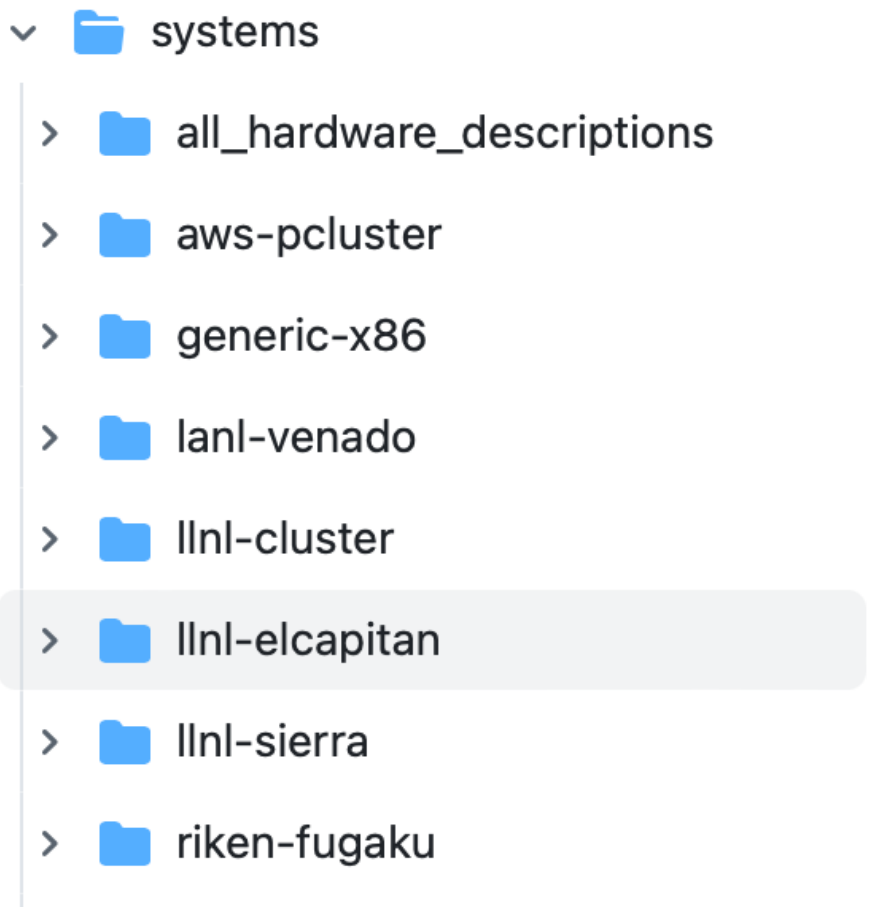
Full specification enables reproducibility, replicability, and automation

HPC System definition for *Performance*

- Resources (CPU cores, GPUs)
- Scheduler (Slurm, flux)
- Process mapping (cores, sockets, GPUs, NICs)
- On-node parallelism (CUDA, ROCM, OpenMP)
- Software stack
  - Compilers (which is best?)
  - MPI implementations (best?)
  - Math libraries

Goal: Use the system correctly

HPC System in Benchpark: *Specify Once*




```
class LlnlElcapitan(System):
    variant(
        "rocm",
        default="5.5.1",
        values=("5.4.3", "5.5.1", ”6.2.4"),
        description="ROCm version",
    )

    variant(
        "compiler",
        default="cce",
        values=("gcc", "cce", "rocmcc"),
        description="Which compiler to use",
    )

    def initialize(self):
        super().initialize()
        self.scheduler = "flux"
        self.sys_cores_per_node = "128"
```

benchpark system init --dest=elcap llnl-elcapitan rocm=6.2.4 compiler=cce

System Specifications

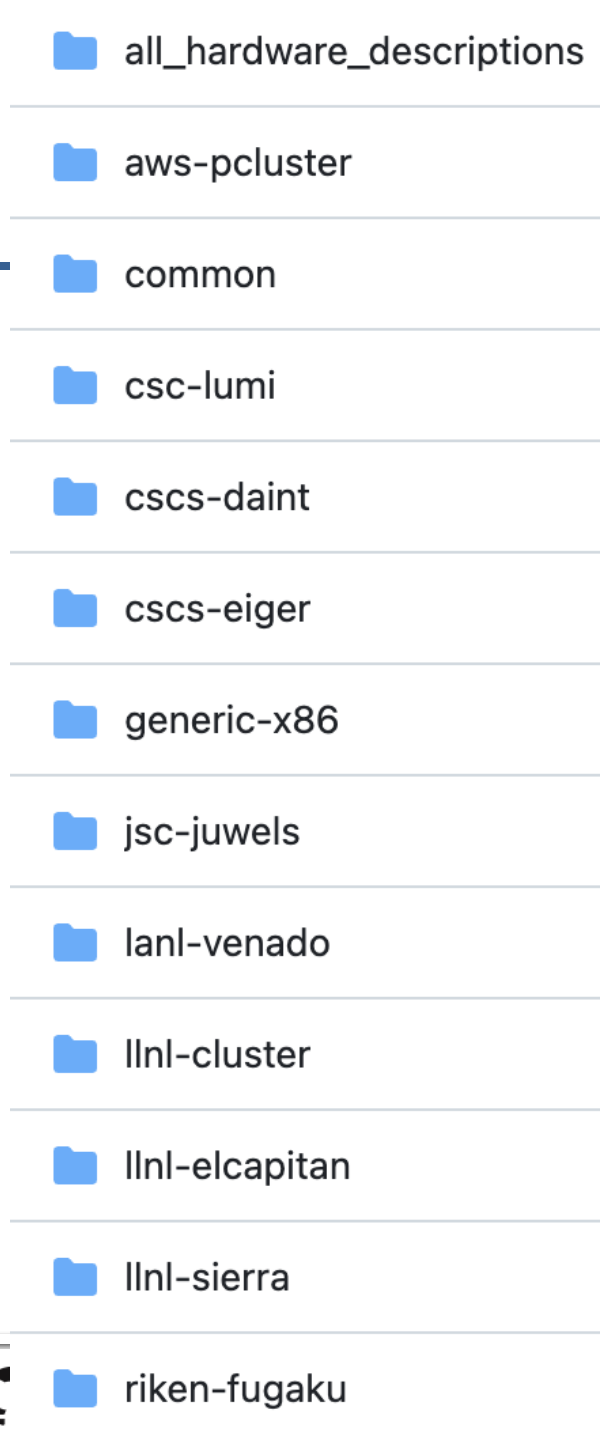
The table below provides a directory of information for systems that have been specified in Benchpark. The column headers in the table below are available for use as the `system` parameter in `benchpark setup`.

Search:

name	integrator.vendor	integrator.name	processor.vendor	processor.name	processor.ISA	processor.u
Atos-zen2-A100-Infiniband	Atos	XH2000	AMD	EPYC-Zen2	x86_64	zen2
AWS_PCluster-zen-EFA	AWS	ParallelCluster	AMD	EPYC-Zen	x86_64	zen
DELL-cascadelake-InfiniBand	DELL		Intel	Xeon6248R	x86_64	cascadelake
DELL-sapphirerapids-OmniPath	DELLEMC	PowerEdge	Intel	XeonPlatinum8480	x86_64	sapphirerapi
Fujitsu-A64FX-TofuD	Fujitsu	FX1000	Fujitsu	A64FX	Armv8.2-A-SVE	aarch64
HPECray-haswell-P100-Infiniband	HPECray		Intel	Xeon-E5-2650v3	x86_64	haswell
HPECray-neoverse-H100-Slingshot	HPECray	EX254n	NVIDIA	Grace	Armv9	neoverse
HPECray-zen2-Slingshot	HPECray		AMD	EPYC-7742	x86_64	zen2
HPECray-zen3-MI250X-Slingshot	HPECray	EX235a	AMD	EPYC-Zen3	x86_64	zen3
HPECray-zen4-MI300A-Slingshot	HPECray	EX255a	AMD	EPYC-Zen4	x86_64	zen4
IBM-power9-V100-Infiniband	IBM	AC922	IBM	POWER9	ppc64le	power9
Penguin-icelake-OmniPath	PenguinComputing	RelionCluster	Intel	XeonPlatinum924248C	x86_64	icelake
Supermicro-icelake-OmniPath	Supermicro		Intel	XeonPlatinum8276L	x86_64	icelake
x86_64					x86_64	

Systems in Benchpark: July 2025

- 4 in Europe
- 6 in US labs
- 1 in Japan
- 1 at a university
- 2 Cloud systems



HPC Experiment definition for *Performance*

- On-node parallelism (CUDA, ROCM, OpenMP)
- Problem sizes
  - Overall problem size, or
  - Per node or per GPU
- Scaling studies
  - How to scale
  - How to decompose
- Resources (cores, GPUs)

Goal: Specify reproducible sets of experiments that map onto specific Systems

HPC Experiment in Benchpark: *Specify Once*

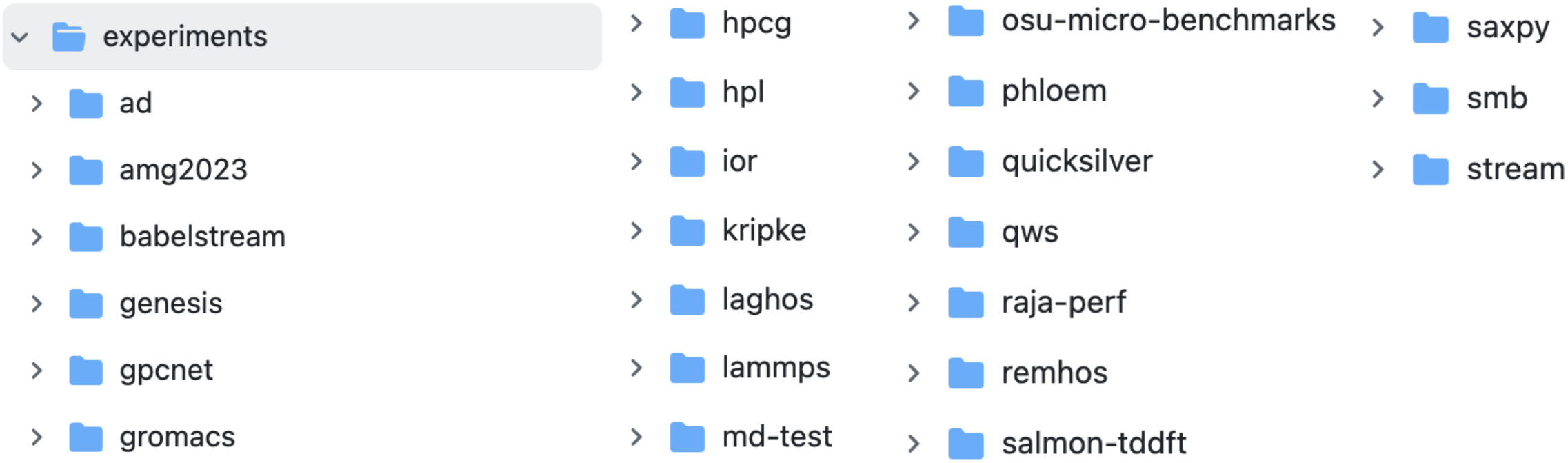
```
class Amg2023(Experiment, OpenMPExperiment, CudaExperiment, ROCmExperiment,
        Scaling(ScalingMode.Strong, ScalingMode.Weak, ScalingMode.Throughput,
                Caliper):
  variant(
    "workload",
    default="problem1",
    values=("problem1", "problem2"),
    description="problem1 or problem2",
  )

  def compute_applications_section(self):
        self.register_scaling_config({
            ScalingMode.Strong: {
                “n_resources”: lambda var, itr, dim, scaling_factor: var.val(dim) * scale,
                “problem_size”: lambda var, itr, dim, scaling_factor: var.val(dim) // scale,
            },
            ScalingMode.Weak: {
                “n_resources”: lambda var, itr, dim, scaling_factor: var.val(dim) * scale,
                “problem_size”: lambda var, itr, dim, scaling_factor: var.val(dim),
            },
            ScalingMode.Throughput: {
                “n_resources”: lambda var, itr, dim, scaling_factor: var.val(dim),
                “problem_size”: lambda var, itr, dim, scaling_factor: var.val(dim) * scale,
```

amg2023 +rocm scaling=strong workload=problem2 caliper=mpi,time

Benchmarks and Experiments

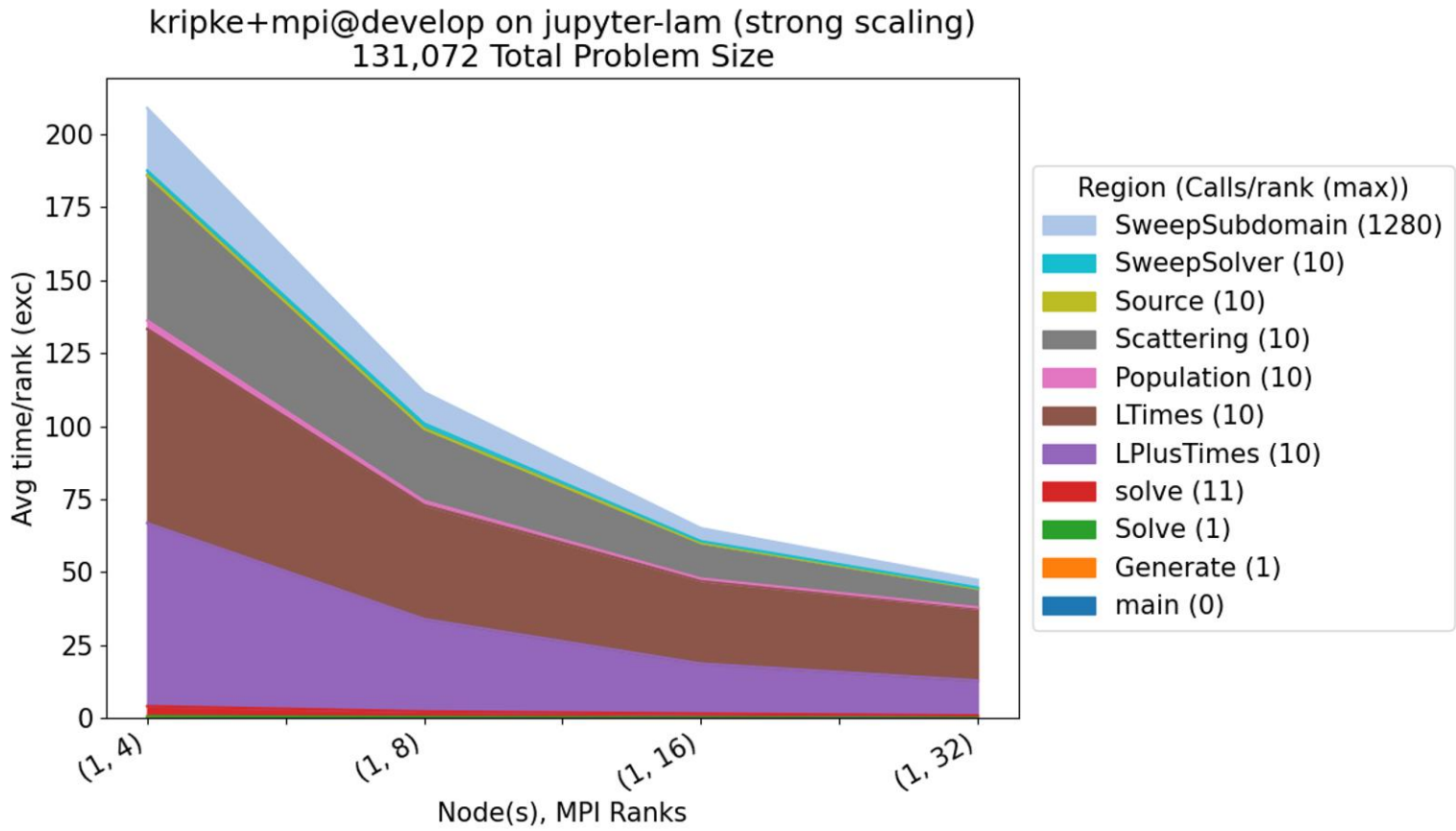
	ad	amg2023
application-domain	[]	['engineering']
benchmark-scale	[]	['large-scale', 'multi-node', 'single-node', 'sub-node']
communication	['mpi']	['mpi']
communication-performance-characteristics	[]	['network-collectives', 'network-latency-bound']
compute-performance-characteristics	[]	['high-branching', 'mixed-precision']
math-libraries	[]	['hypre']
memory-access-characteristics	[]	['high-memory-bandwidth', 'irregular-memory-access', 'large-memory-footprint', 'regular-m
mesh-representation	[]	['block-structured-grid']
method-type	['compiler-transformation']	['solver', 'sparse-linear-algebra']
programming-language	['c', 'c++']	['c']
programming-model	[]	['openmp', 'cuda', 'rocm']
instrumented-caliper	False	True
scaling-experiments	[]	['strong', 'weak']

Experiments in Benchpark: July 2025



- HPL, HPCG
- 4 microbenchmarks
- 3 MPI benchmarks
- 8 US
- 1 Europe
- 2 Japan

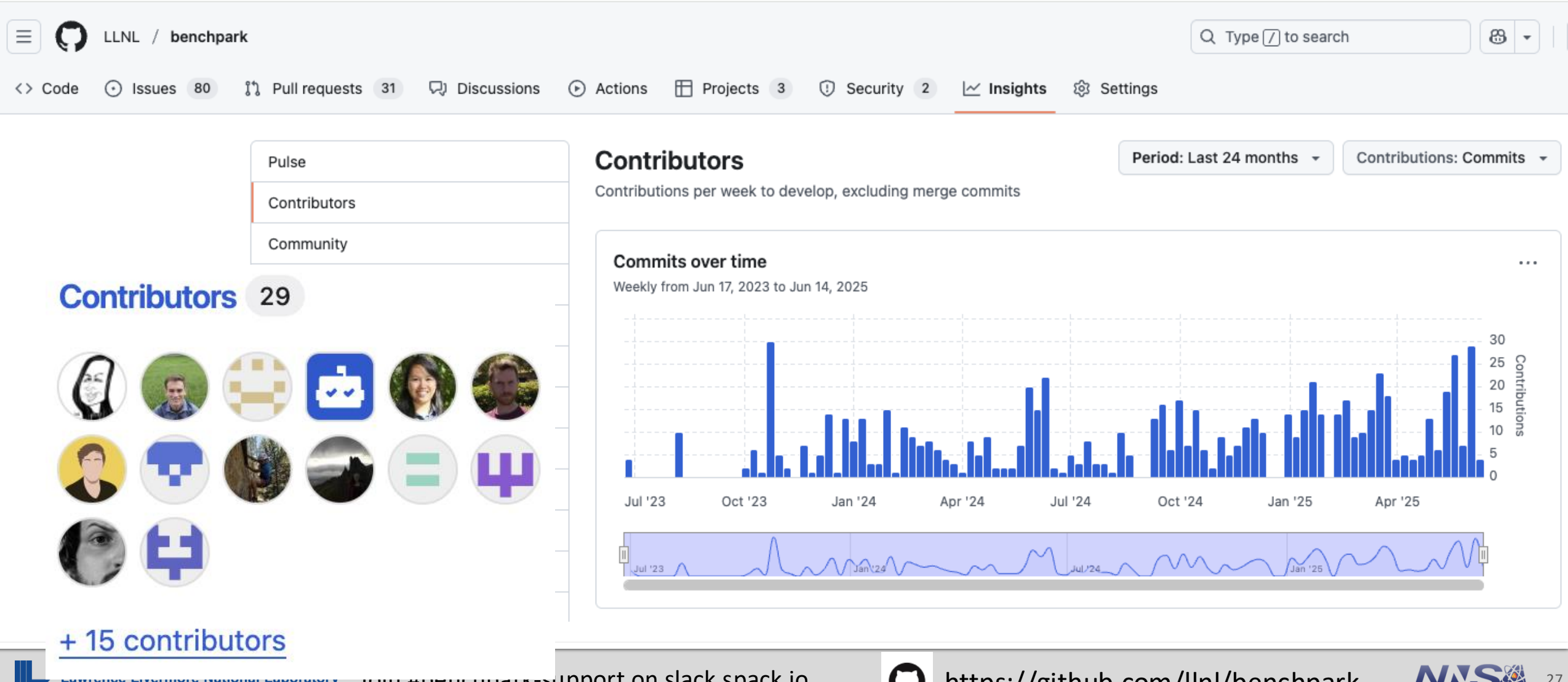
`benchpark analyze` for generating pre-defined analysis charts




```
main
├─ Generate
│  ├─ MPI_Allreduce
│  ├─ MPI_Comm_split
│  └─ MPI_Scan
├─ MPI_Allreduce
├─ MPI_Bcast
├─ MPI_Comm_dup
├─ MPI_Comm_free
├─ MPI_Comm_split
├─ MPI_Finalize
├─ MPI_Finalized
├─ MPI_Gather
├─ MPI_Get_library_version
├─ MPI_Initialized
└─ Solve
   └─ solve
      ├─ LPlusTimes
      ├─ LTimes
      ├─ Population
      │  └─ MPI_Allreduce
      ├─ Scattering
      ├─ Source
      └─ SweepSolver
         ├─ MPI_Irecv
         ├─ MPI_Isend
         ├─ MPI_Testany
         ├─ MPI_Waitall
         └─ SweepSubdomain
```

See https://software.llnl.gov/benchpark/benchpark-analyze.html

Contributions from 11 orgs (60% non-LLNL)



Benchpark codifies benchmarking steps

- Benchpark does **not** replace benchmark source, build system, or Spack package
- Benchpark manages benchmark **experiments** and how they map onto **systems** with specified (or default)
  - Compilers
  - MPI/comm libraries
  - Math libraries
- Start running benchmark experiments on your system with just a few commands

```
git clone git@github.com:LLNL/benchpark.git
```

```
benchpark list systems
benchpark system init --dest=elcap llnl-elcapitan
```

```
benchpark list experiments
benchpark experiment init dest=amg
                          amg2023+rocm+strong
```

```
benchpark setup amg elcap workspace
ramble workspace setup
```

```
ramble on
```

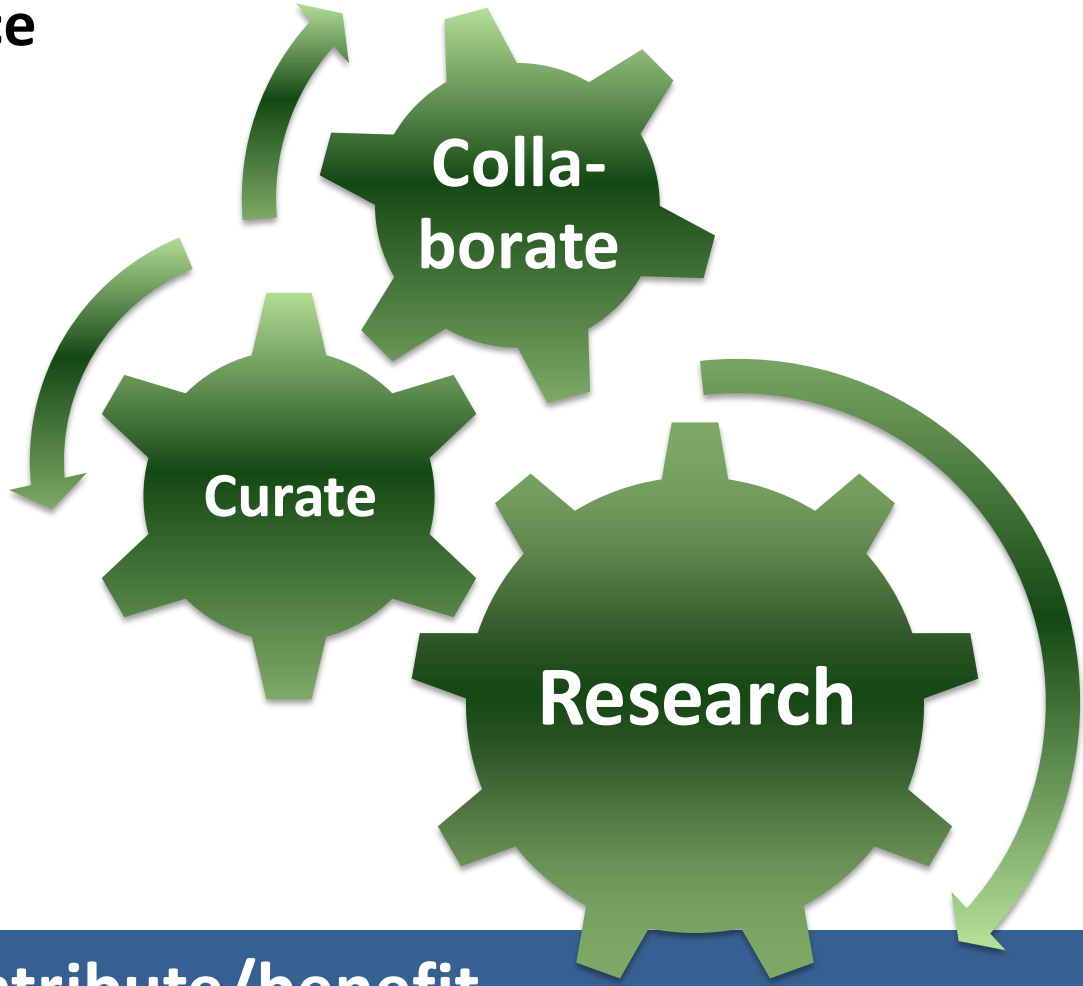
Who can use Benchpark

People who want to use or distribute benchmarks for HPC!

1. **End Users of HPC Benchmarks**
   - Install, run, analyze performance of HPC benchmarks
2. **Benchmark Developers**
   - People who want to share their benchmarks
3. **Procurement teams at HPC Centers**
   - Curate workload representation, evaluate and monitor system progress
4. **HPC Vendors**
   - Understand the curated workload of HPC centers, propose systems

Catalogued library of *working* benchmarks

- **Enables exploration of large configuration space**
  - —Architectural
  - —Software stack
  - —Temporal
- What architecture and system configuration is best for *my benchmark*?
- On *my system*, is OpenMPI good enough?
- What purpose will Benchpark help you address?



Building a community to contribute/benefit

Benchpark: Open collaborative repository for *reproducible* specifications of HPC benchmarks

Infrastructure-as-code benchmark specification enables **reproducibility, replicability, and automation**

- HPC Systems
- HPC Experiments

Specify	Run	Reproduce	Replicate	Maintain	Record
• How to build and run benchmarks on a system	• Run an experiment on a system	• Re-run an experiment on a system	• Run an experiment on a new system	• CI: Run experiment on HPC systems	• Perf.measu-rements + *full spec* of experiment

- Tagging, keywords for publications
- Performance metrics, metrics of usefulness
- Dashboards: Archive specs+results

- CI pipelines on PRs from GitHub at data centers across the world and in the cloud

*Olga Pearce et al, Continuous Benchmarking, HPCTests SC|23

Full specification enables reproducibility, replicability, and automation

Benchpark roadmap and community engagement

Future directions:

- Suite curation: Reproducible specification of an entire suite
- Tagging: Keywords for publications, finding benchmarks
- Metrics: Performance, usefulness
- Dashboards: Archive+share *specs*+results, Slices in configuration space
- CI pipelines at data centers across the world and in the cloud

Community Engagement:

- Co-design / vendors on board, but incentive for app teams? (carrot or stick?)
- Who owns which parts of the specification and approves changes?
- Who finances R&D and maintenance?
- ROI for the people working on it? → think about post docs, researchers, etc.

Collaboration, reproducibility, fully specified public results

Benchpark Tutorial Materials

Find these slides and associated scripts here:

software.llnl.gov/benchpark/example-workflow.html

We also have a channel on Spack slack.
You can join here:

slack.spack.io

Join the **#benchpark-support** channel!

You can continue to ask questions here after the conference has ended.

Tutorial Instances: http://bit.ly/4kGQDlc

- We have an AWS instance for the hands-on component of this tutorial
- The instance provides:
  - Pre-installed Thicket, Caliper, and Benchpark and required dependencies
  - Caliper source code demos
  - Thicket Jupyter notebooks and datasets for performance analysis

When logging in to the instance:

- Please use a unique username to avoid resource allocation conflicts
  - First initial followed by last name (e.g., jdoe)
- PW: hpctutorial25

Join us after HPDC!

- All tutorial materials
  - software.llnl.gov/Caliper
  - thicket.readthedocs.io/en/latest/tutorial_materials.html
  - software.llnl.gov/benchpark/example-workflow.html
- Connect with us on Spack slack

  #benchpark-support

- We want your feedback!

Thank you!

slack.spack.io
#benchpark-support

★ Star us on GitHub!

github.com/llnl/caliper

github.com/llnl/thicket

github.com/llnl/benchpark

CASC

Center for Applied
Scientific Computing